

Lecture 1: Introduction

DM571 DM889: Software Engineering

Francesco Daghero

fdag@mmmi.sdu.dk

Tiziano Santilli

tisa@mmmi.sdu.dk

Who are we

Tiziano Santilli

tisa@mmmi.sdu.dk

Postdoctoral researcher

Software Engineering, Software
Architecture, LLMs, Compliance

Arm Wrestling, LEGO, pizza

Francesco Daghero

fdag@mmmi.sdu.dk

Postdoctoral researcher

Computer engineering, Machine
Learning, Embedded Systems, LLMs

Cycling, Swimming, Smiling

*For any contact put both in cc

Lectures

- Interactive
- 45 minutes per hour
- Start with theory
- Group works
- Quiz
- Ask questions anytime
- Attending classes in person helps you pass the exam
- Slides online before the lecture

Exercise Sessions

- **Instructor:** Oliver Aleksander Larsen
- Exercises related to the theoretical lecture
- Project work (if time permits)
- Workshop(s)

Project

- Handed out during the course
- Consists of several parts related to the theoretical lectures
- Groups up to 4
- Must be delivered to attend exam
- Can move exam grade with -1, 0, 1
- Will be same structure but different domains for each group

Exam

- **Project must be passed**
- Theoretical and project related questions
- Internal censor
- ~ 25 minutes pr. student
- Graded

Expected learning outcome

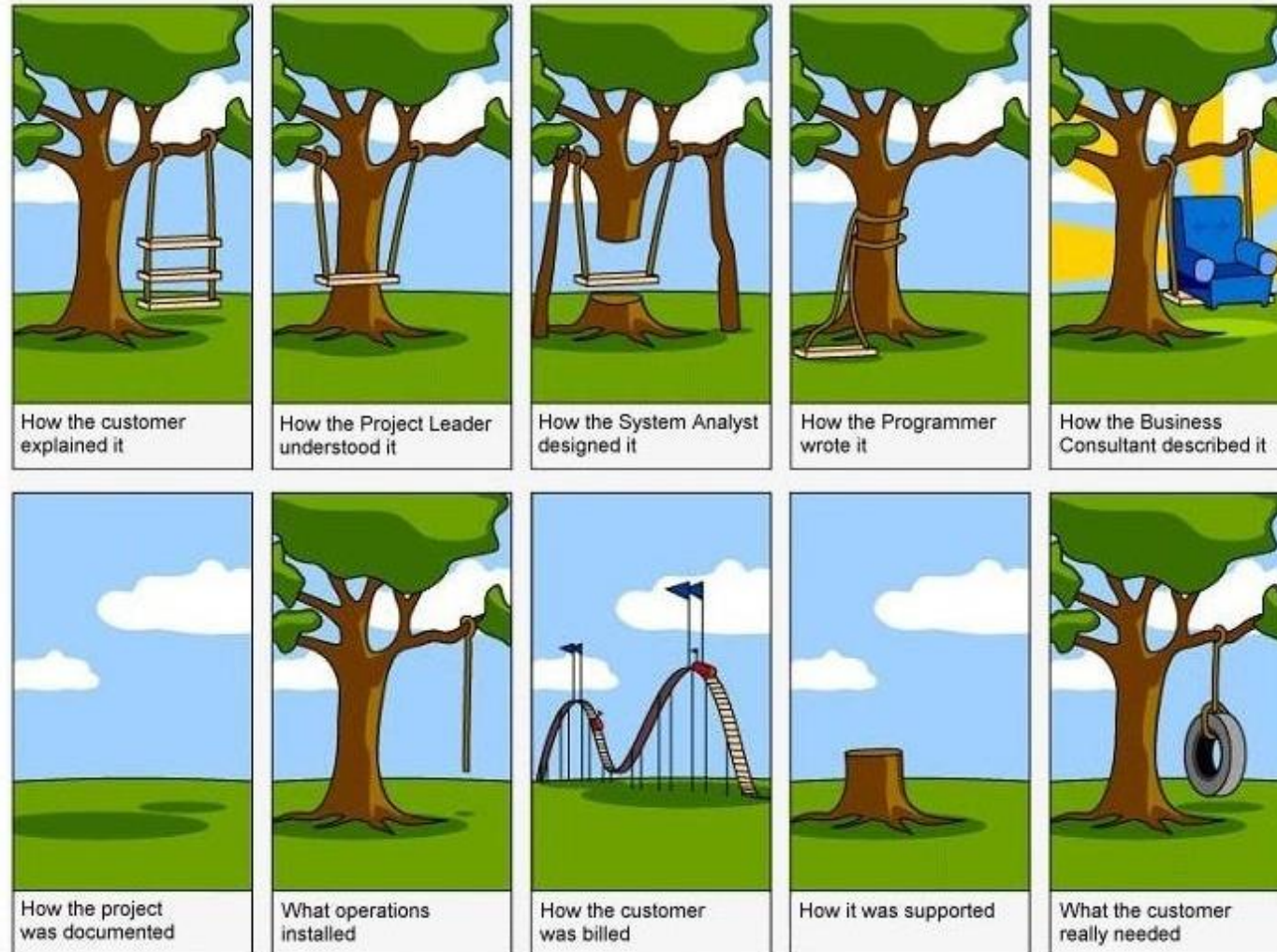
The learning objective of the course is that the student demonstrates the ability to:

- **Understand ABSTRACTION**
- Describe in detail the core activities of the software development process.
- Describe and discuss process models, and perform substantiated choices between these.
- Describe tools and modeling formalisms from the course.
- Describe and discuss software patterns and other architectural structures, and perform substantiated choices between these.
- Apply knowledge and methods from the course in concrete settings in programming projects.
- Discuss the influence and impact of software on humans, organizations, and society, including ethical issues.

Use of AI

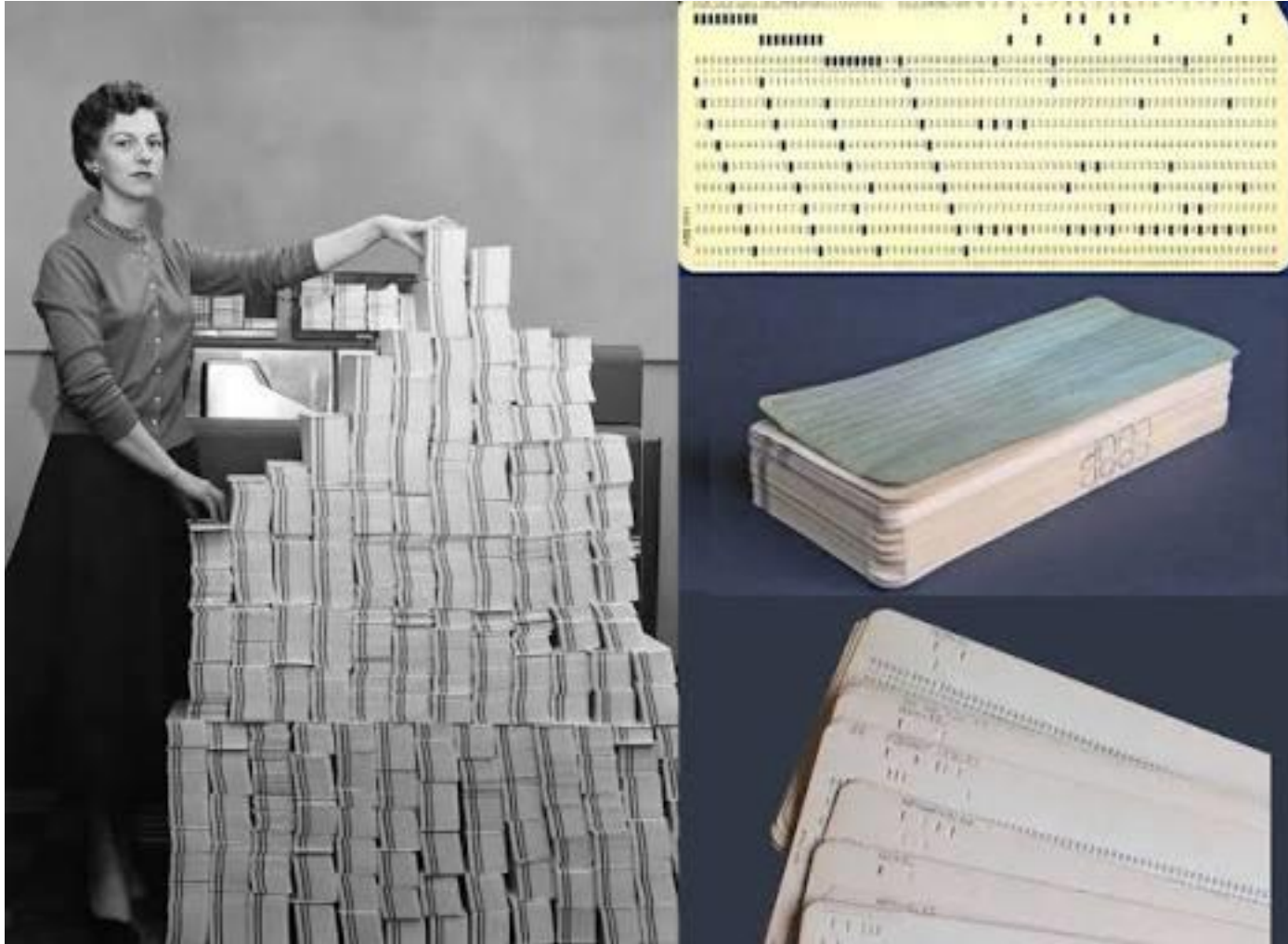
This course in pills:

Build the Right Thing (Effectiveness) and **Build the Thing Right (Efficiency)**



How we started?

Punched cards



Each hole corresponds to a letter, a number, or a symbol.

Pre 1968: Software Crisis

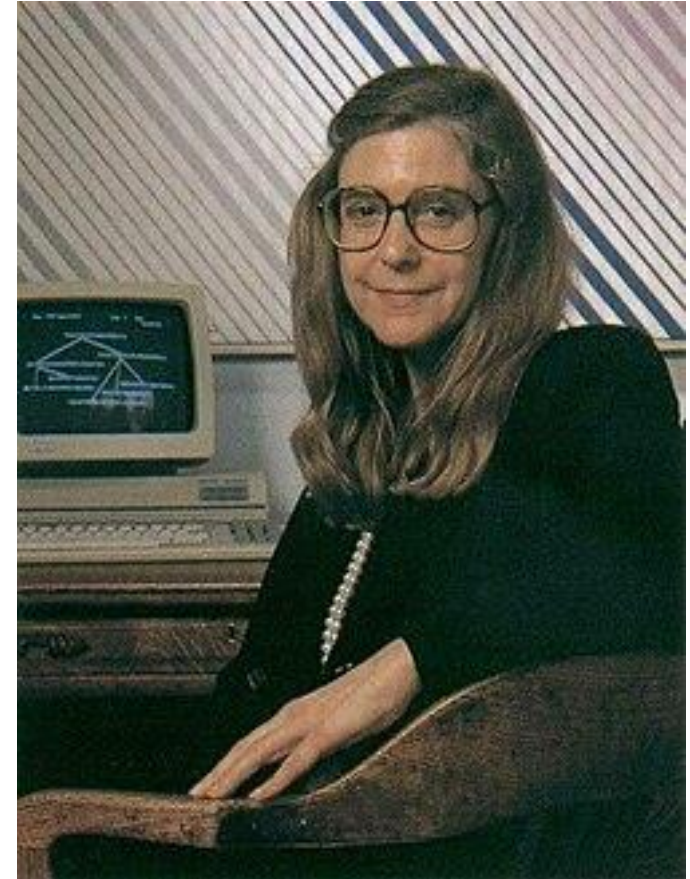
- No formal definition for software engineering
- Hardware-Software Asymmetry
- Severe Project Failures
- Reliability and Maintainability Deficits
- Lack of Process Rigor



1968 Software Engineering

- Margaret realized that there was the need to treat software as an engineering field
- Margaret gave to software (mostly coding at that time) the engineering term

«They liked to kid me about my radical ideas. It was a memorable day when one of the most respected hardware gurus explained to everyone in a meeting that he agreed with me that the process of building software should also be considered an engineering discipline, just like with hardware»



Margaret Hamilton (software engineer)

1970s to 2020s

1970s: Professionalization begins

- Universities start
- ACM & IEEE

1980s: Tools, Methods & Recognition

- PC's
- Commercial software
- Growth of methodologies: Analysis, design, early OOP (Smalltalk, C++)

1990s: Standards & Agile Roots

- ISO/IEC software engineering standards
- Internet, large-scale
- Object Oriented Analysis (UML, ...)

2000s:

- 01 Agile Manifesto
- Scrum, LEAN

2010s:

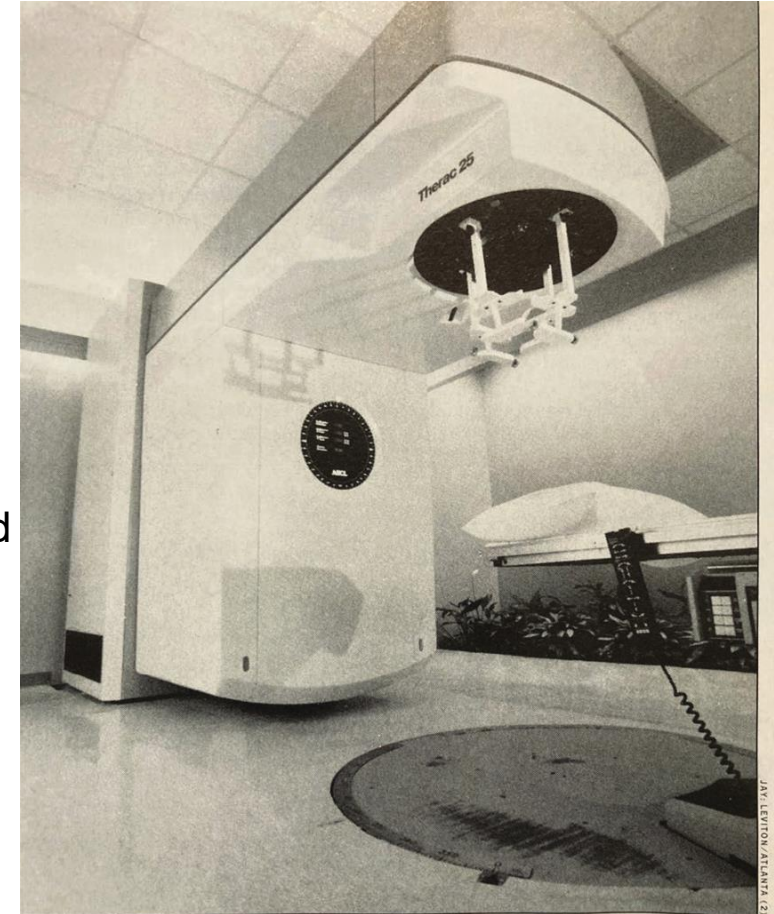
- DevOps, Cloud
- Every company is an IT company

2020s: LLMs, AI, Ethics, Roles

What happens if we don't follow software engineering practices?

Therac-25 (1982)

- Computer-controlled radiation therapy machine
- Multiple patients received radiation overdoses up to 100 times the lethal threshold (**6 direct deaths/permanent injuries**)
- When operators edited treatment modes quickly, the machine kept high-power electron mode with the radiation shield retracted



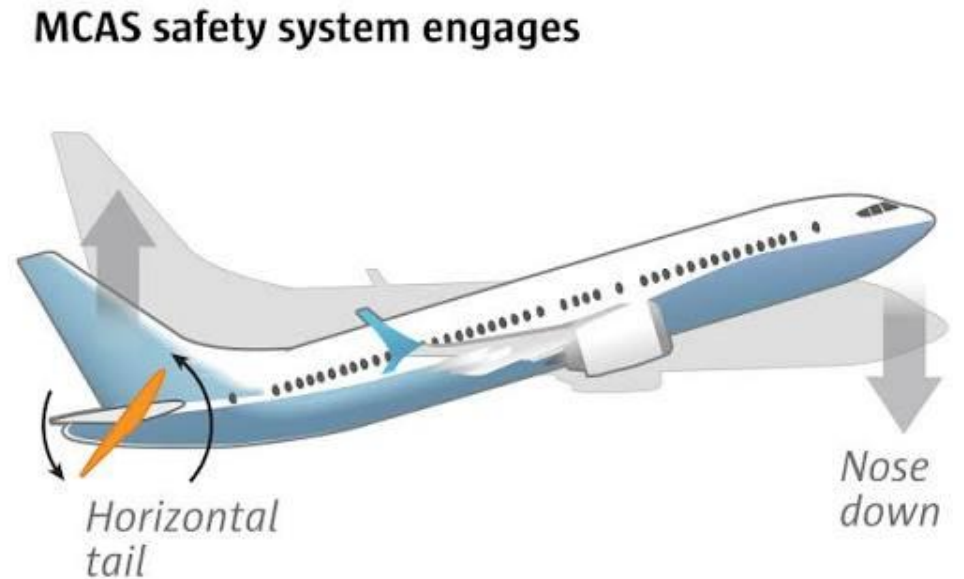
computerized linear accelerator, Emory University's Therac 25 is still available for use on cancer patients.

"Doctors should listen to their patients," says Katy Yarbrough, 63 (with grandsons Robbie, 11 [left], and Richie, 14).

What happens if we don't follow software engineering practices?

Boeing 737 MAX MCAS Disasters (2018–2019)

- Faulty Maneuvering Characteristics Augmentation System (MCAS) software relied on a single Angle-of-Attack (AoA) sensor without cross-validation or hardware redundancy.
- The software repeatedly forced the aircraft's nose down based on bad input, overpowering flight crews
- **346 fatalities**



Seattle Times

What happens if we don't follow software engineering practices?

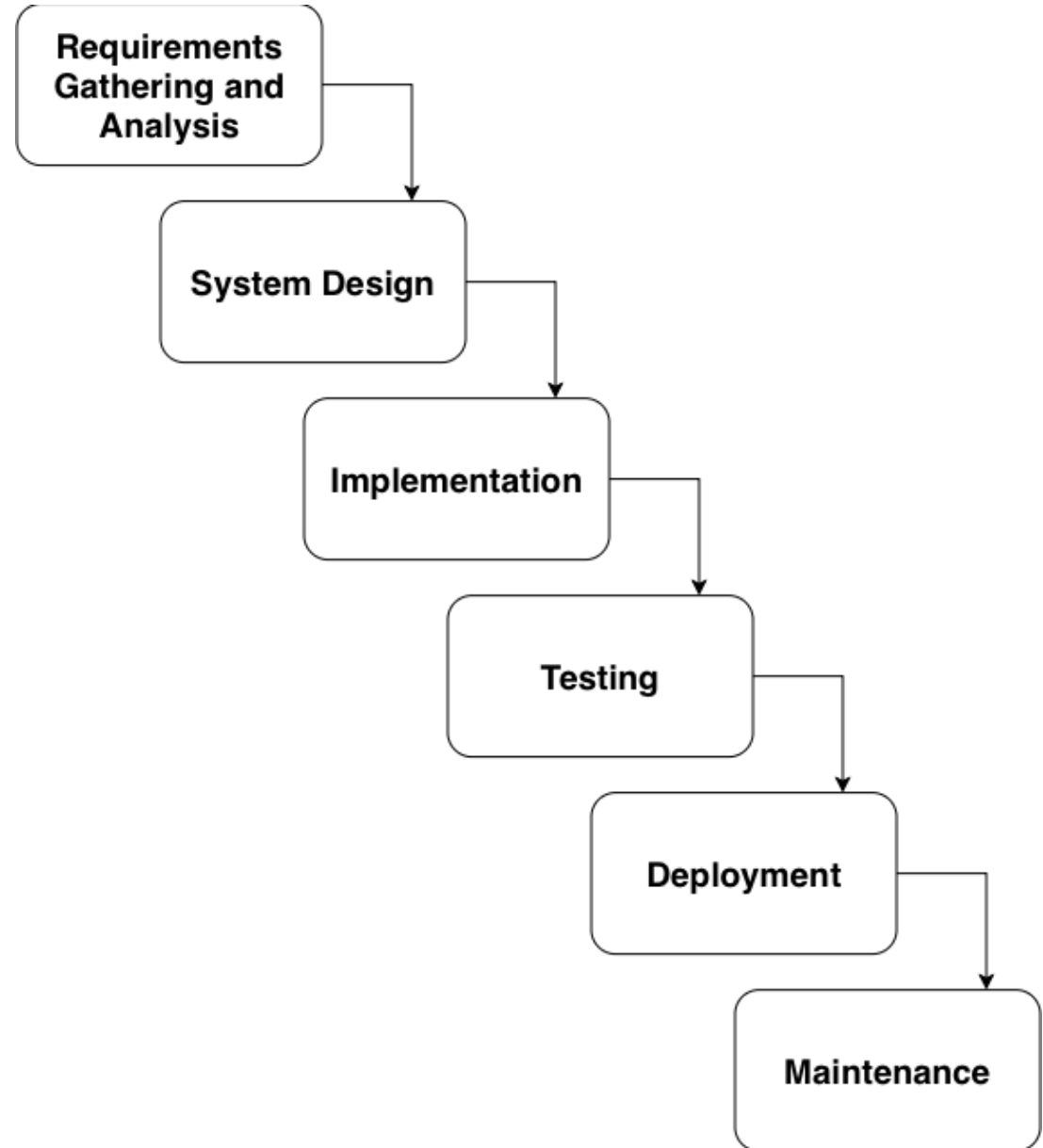
- NASA Mars Climate Orbiter (1999) (\$350 million loss)
- Panama City Oncology Center Overdoses (2000–2001) (8-21 fatalities)
- Knight Capital Group Glitch (2012) (\$440 million lost in 45 minutes)
- Wenzhou train disaster (2011) (40 fatalities)
- Toyota Unintended Acceleration (around 80 fatalities)
- Much more

Seattle Times

Software Development Process

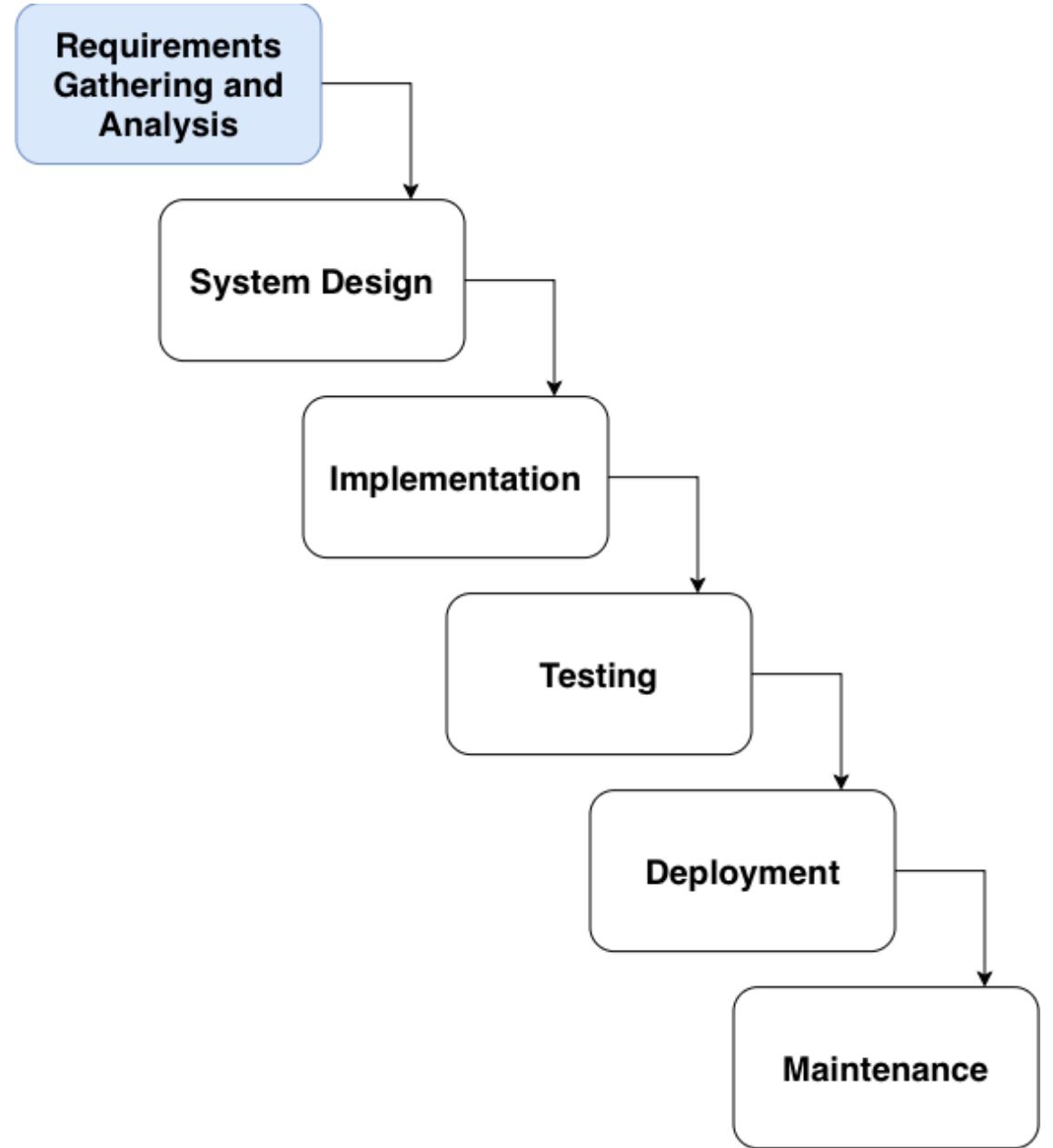
Waterfall Model

- Often credited to Dr. Royce
- Splits development process into phases
- Specialists in each phase
- "Big bang"
- "Obvious approach"
- Sequential



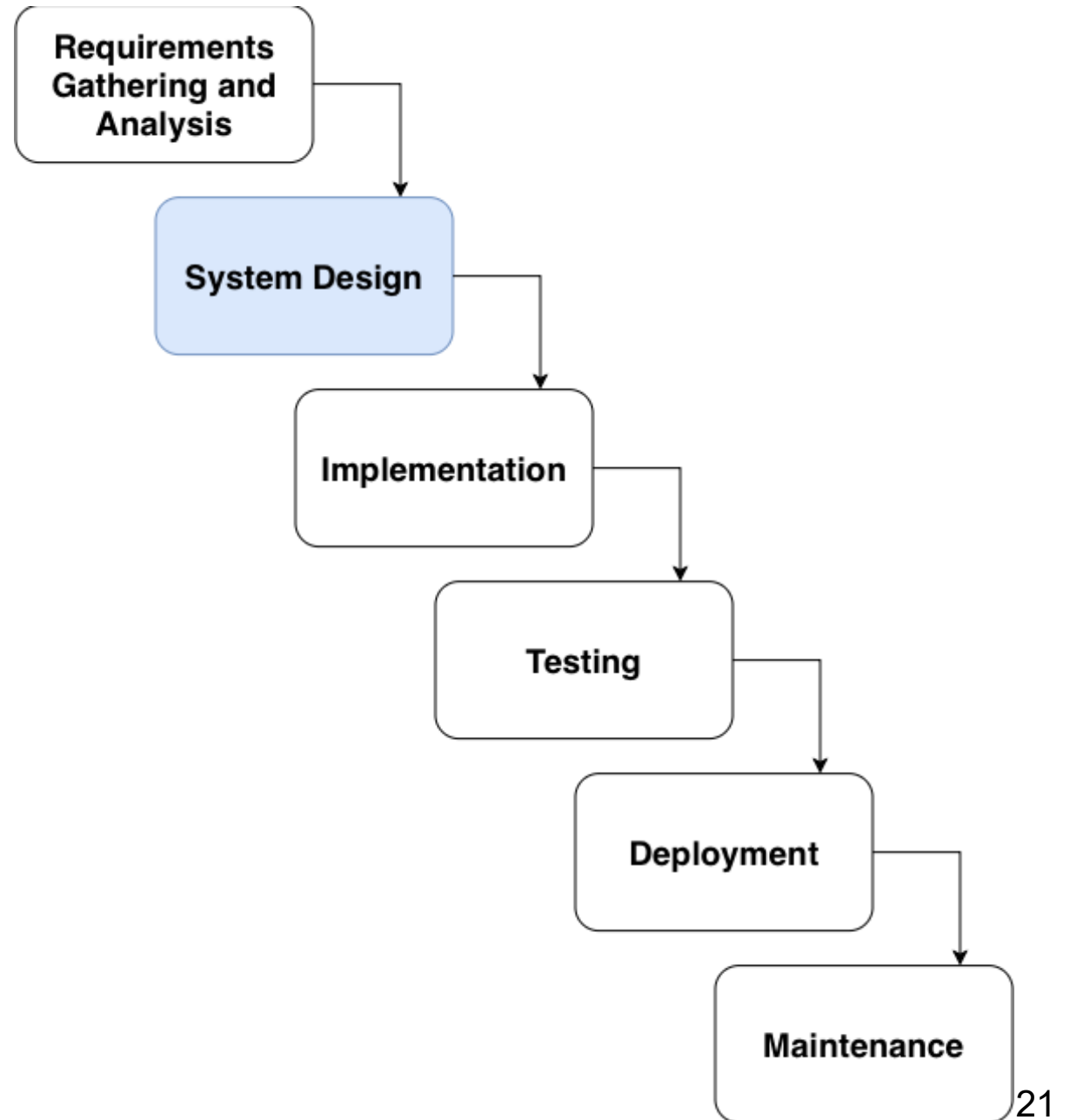
Waterfall Model

- Requirements
- Functional and non-Functional Requirements
- Requirements Prioritization
- Requirements gathering
- Client needs
- Each Functionality is a requirement
- Each system characteristic is a requirement



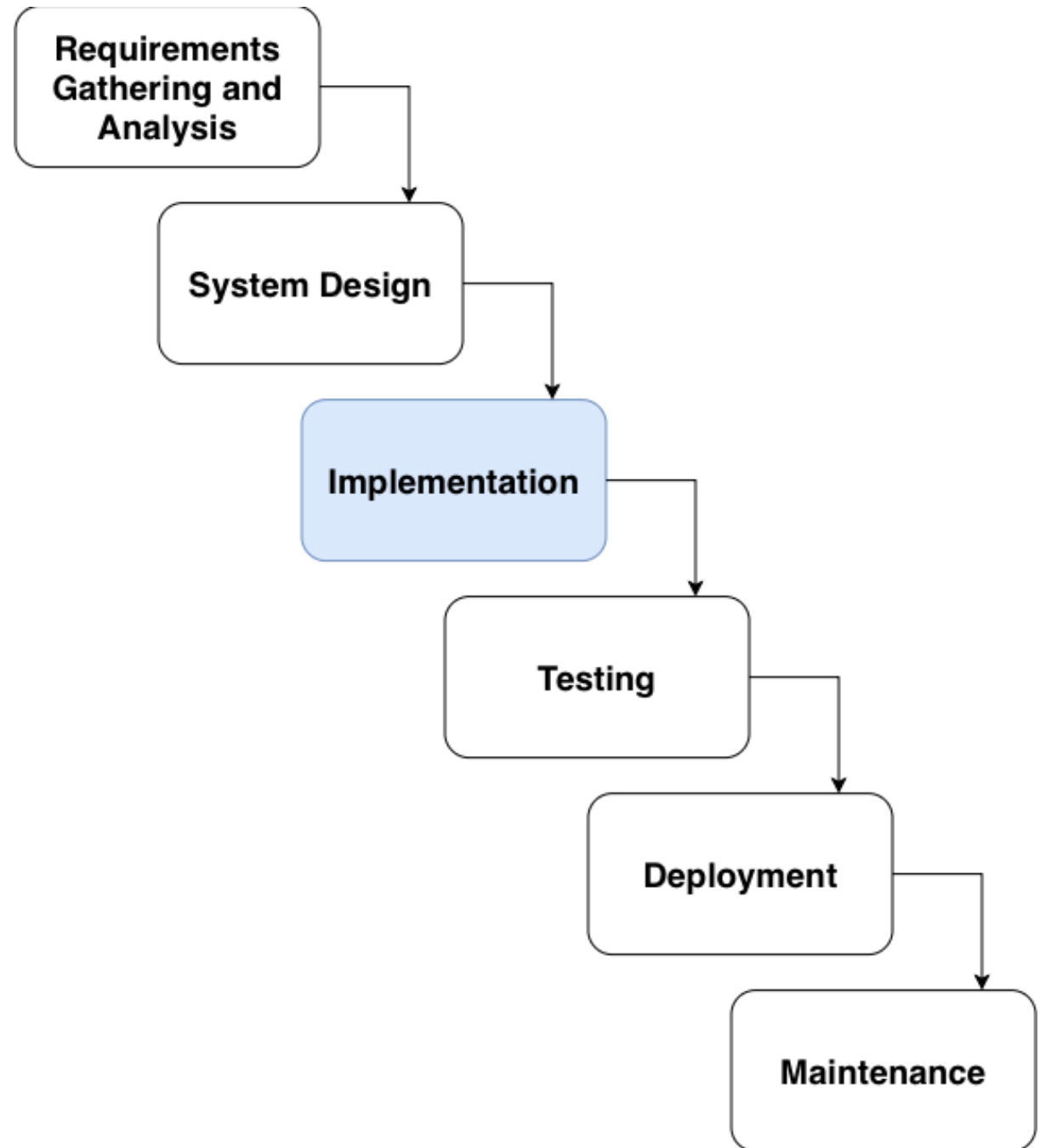
Waterfall Model

- Blueprint of the system
- Software Architecture
- **Abstraction**
- UML
- Stakeholder view
- Data Structure



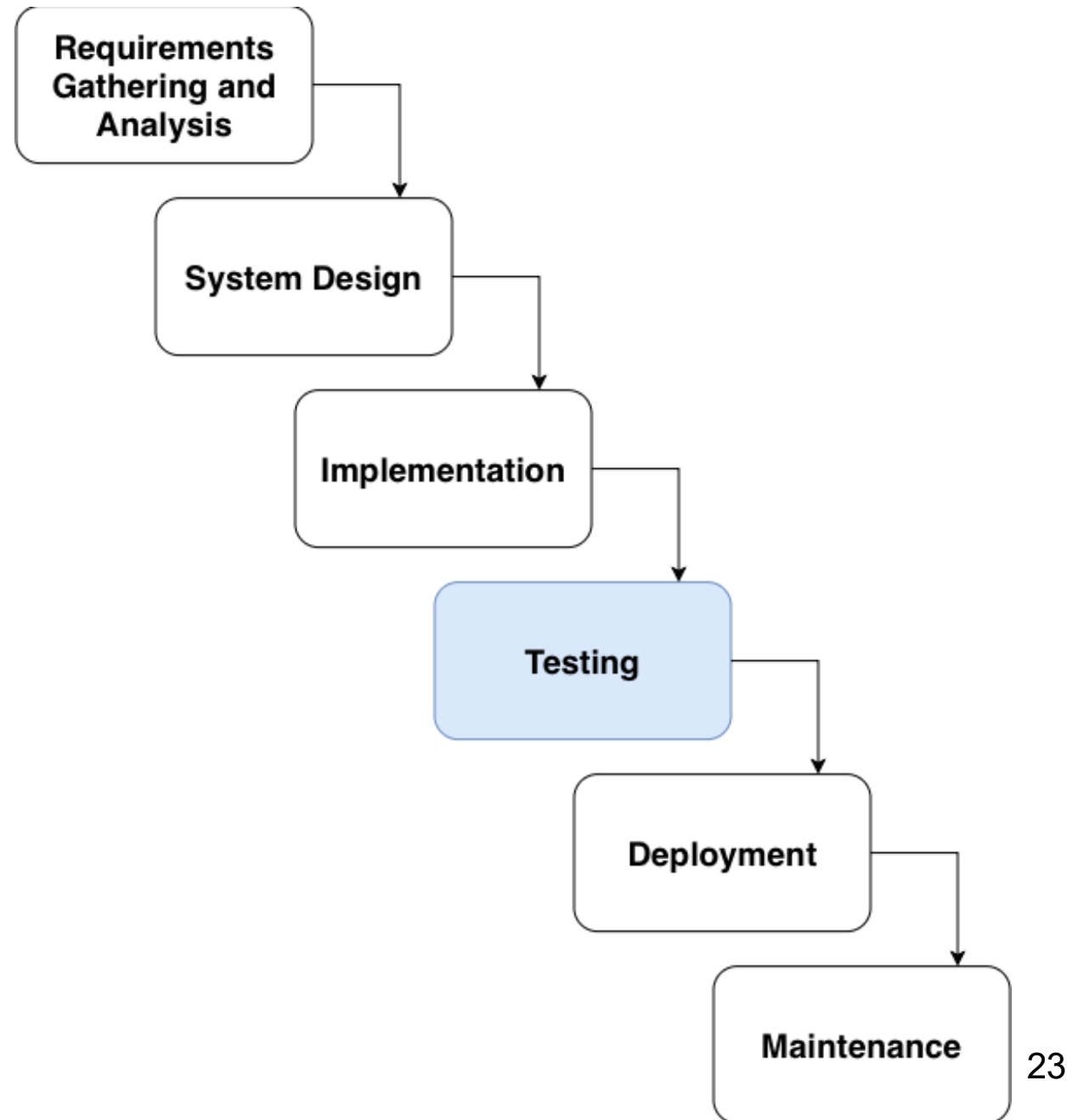
Waterfall Model

- Given System Design and Requirement, Start to code
- “Code Monkeys” or “Code Chickens”
- Programming languages



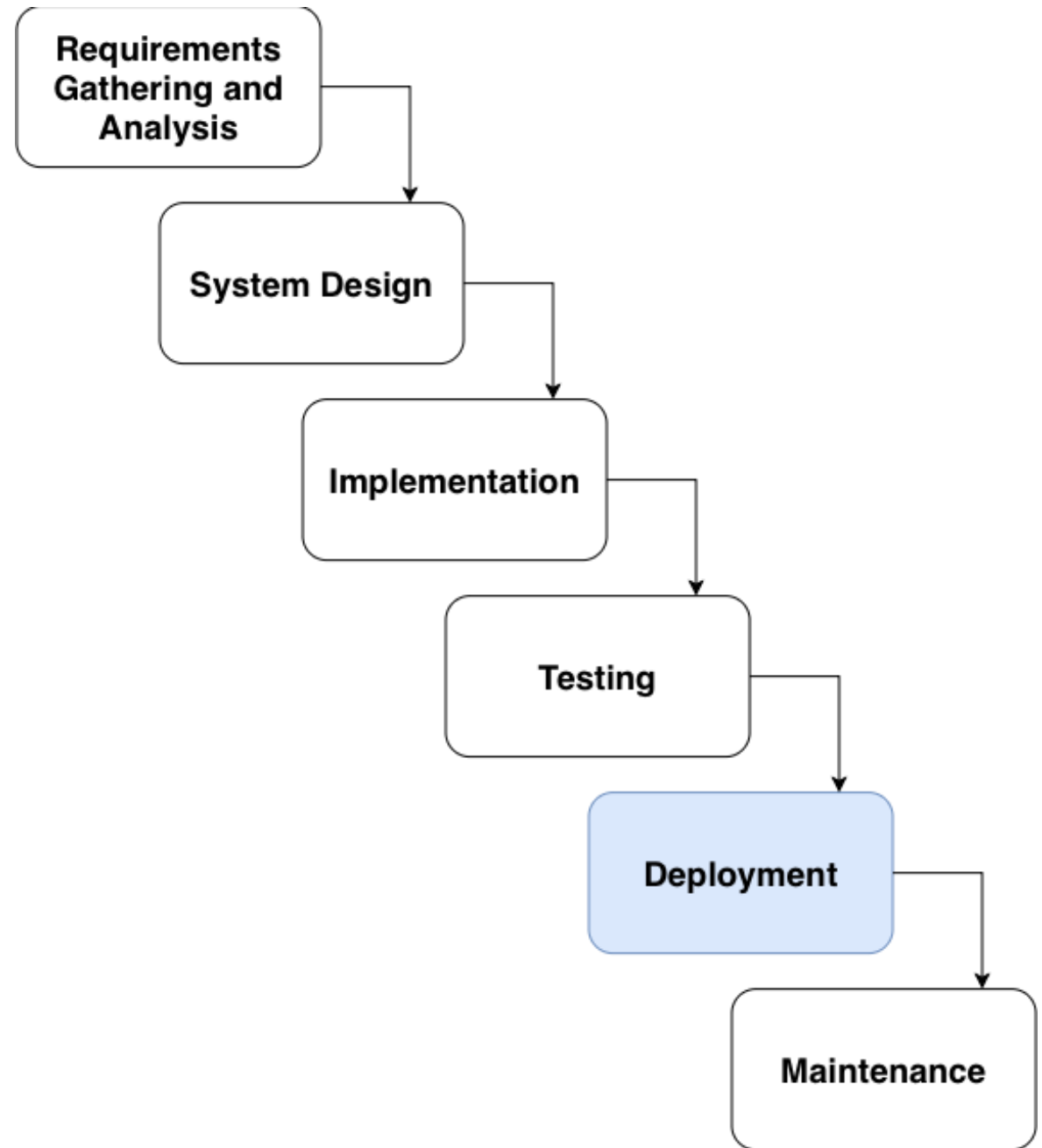
Waterfall Model

- Unit test
- Bottlenecks test
- Validation against Requirements
- Integration Testing
- User Acceptance Testing (UAT)



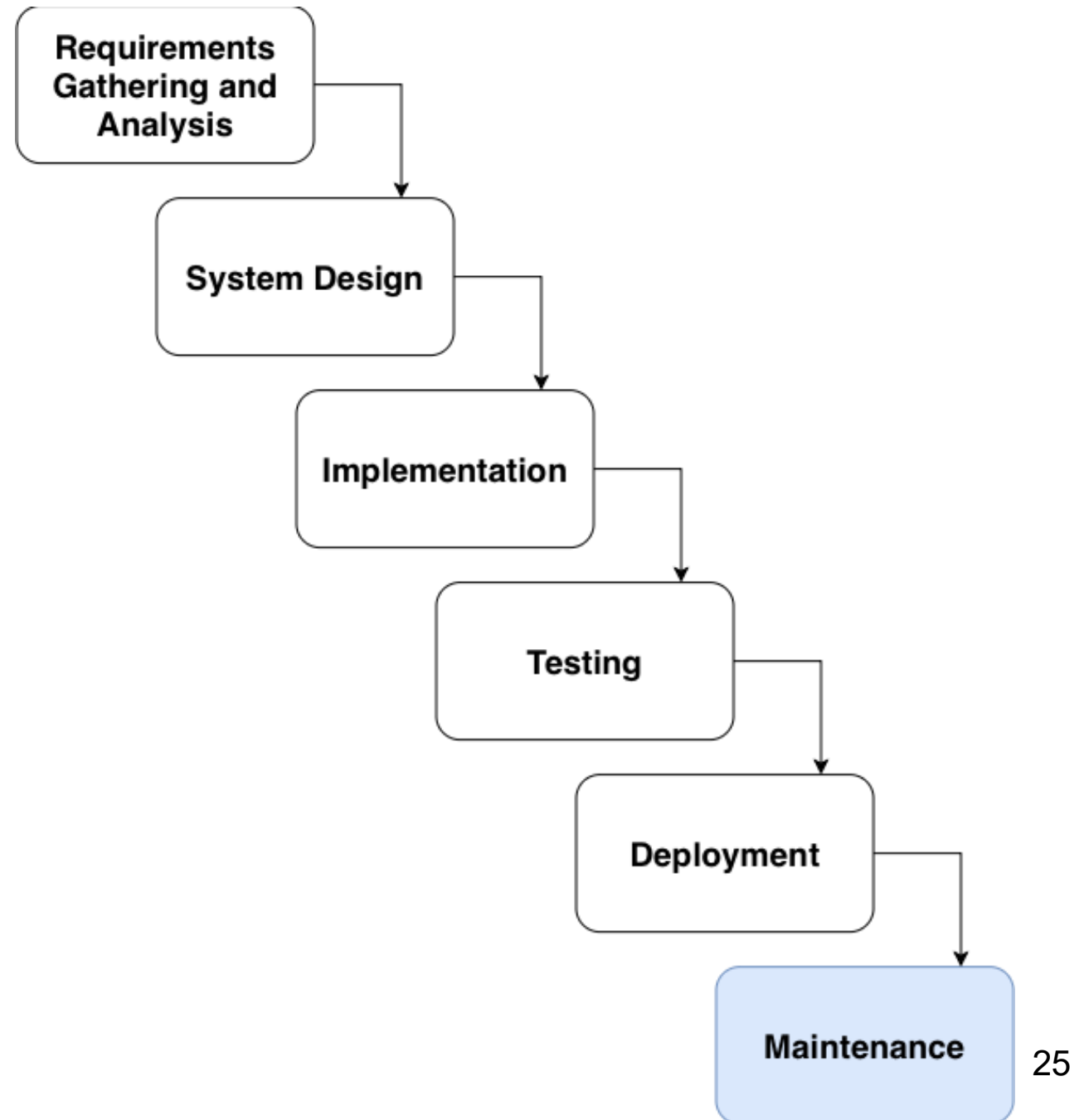
Waterfall Model

- Deploy at customer
- Put in production
- Big Bang

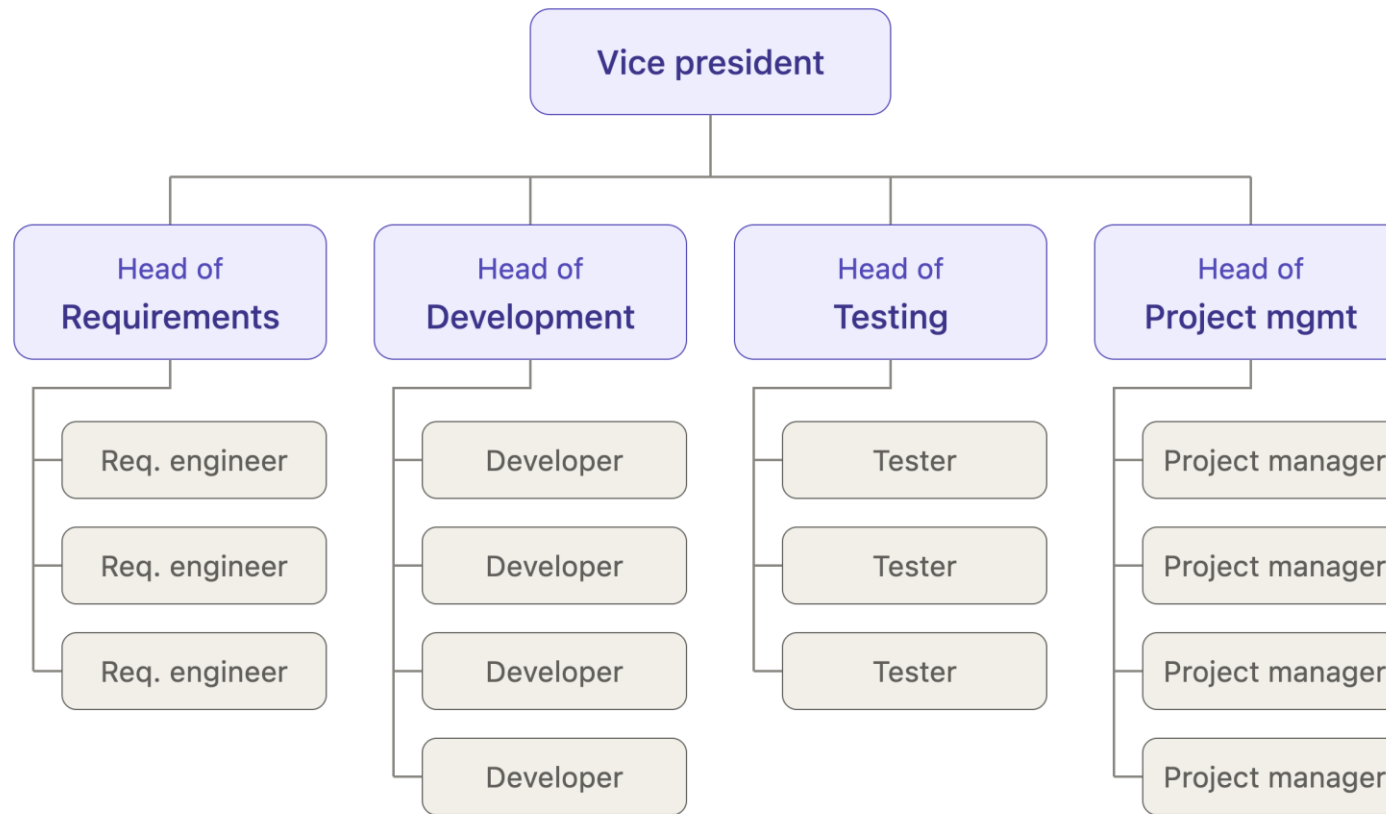


Waterfall Model

- Maintenance if possible
- Usually no updates
- Start new project

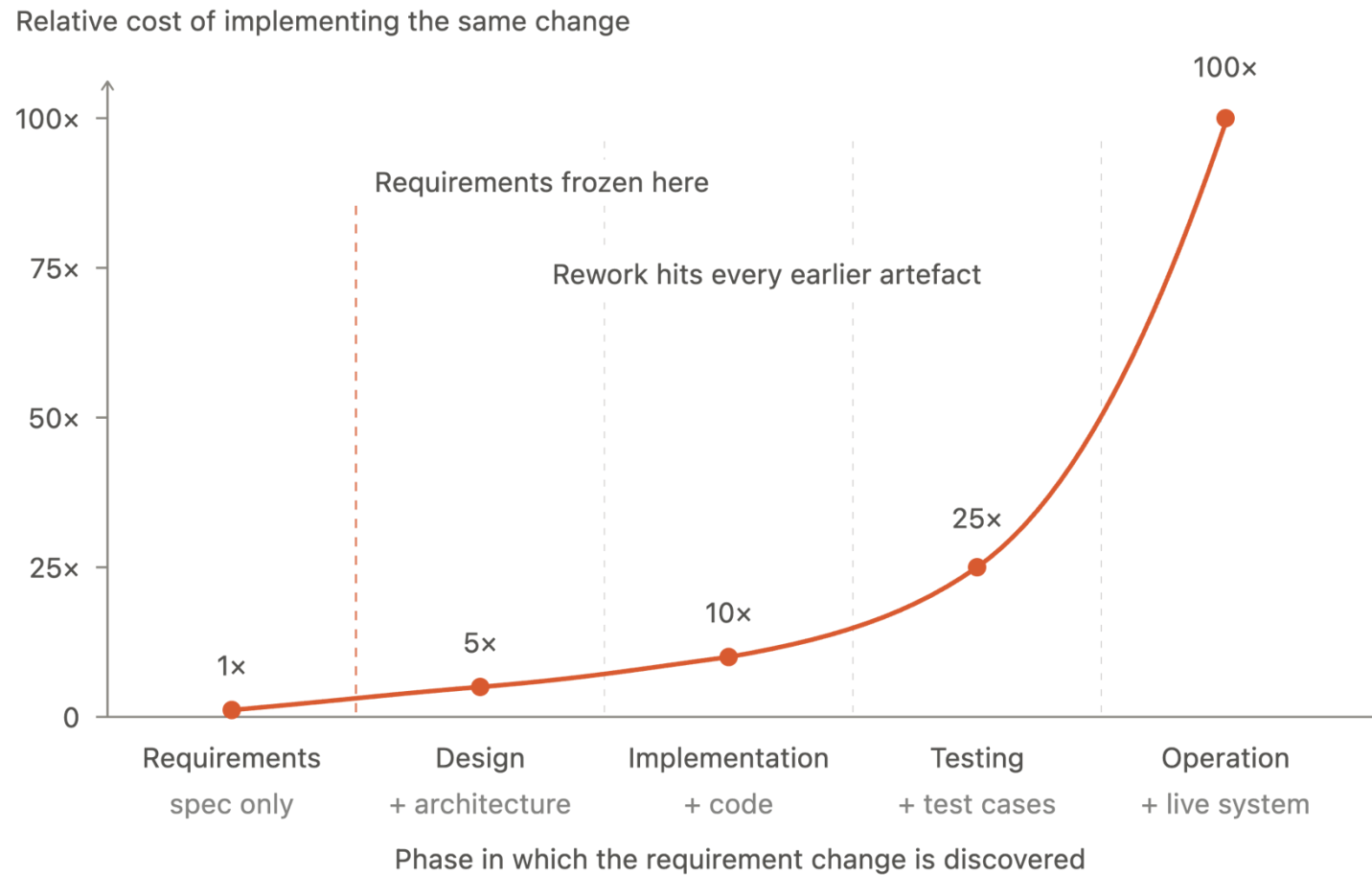


Waterfall Model company organization



Purple = management line, gray = staff

Waterfall Model Cost of Change



Waterfall Model: Pros and Cons

Where is still used?:

- Aerospace, defense
- Healthcare devices
- Embedded system

Pros:

- Traceability and documentation
- Fixed Schedule and Budget
- Clear Project Architecture
- Governance



Cons:

- No change or Update (costly)
- Late User Feedback
- Deployment Risk at the end
- Often skip testing if some phase is late
- Organizational problems

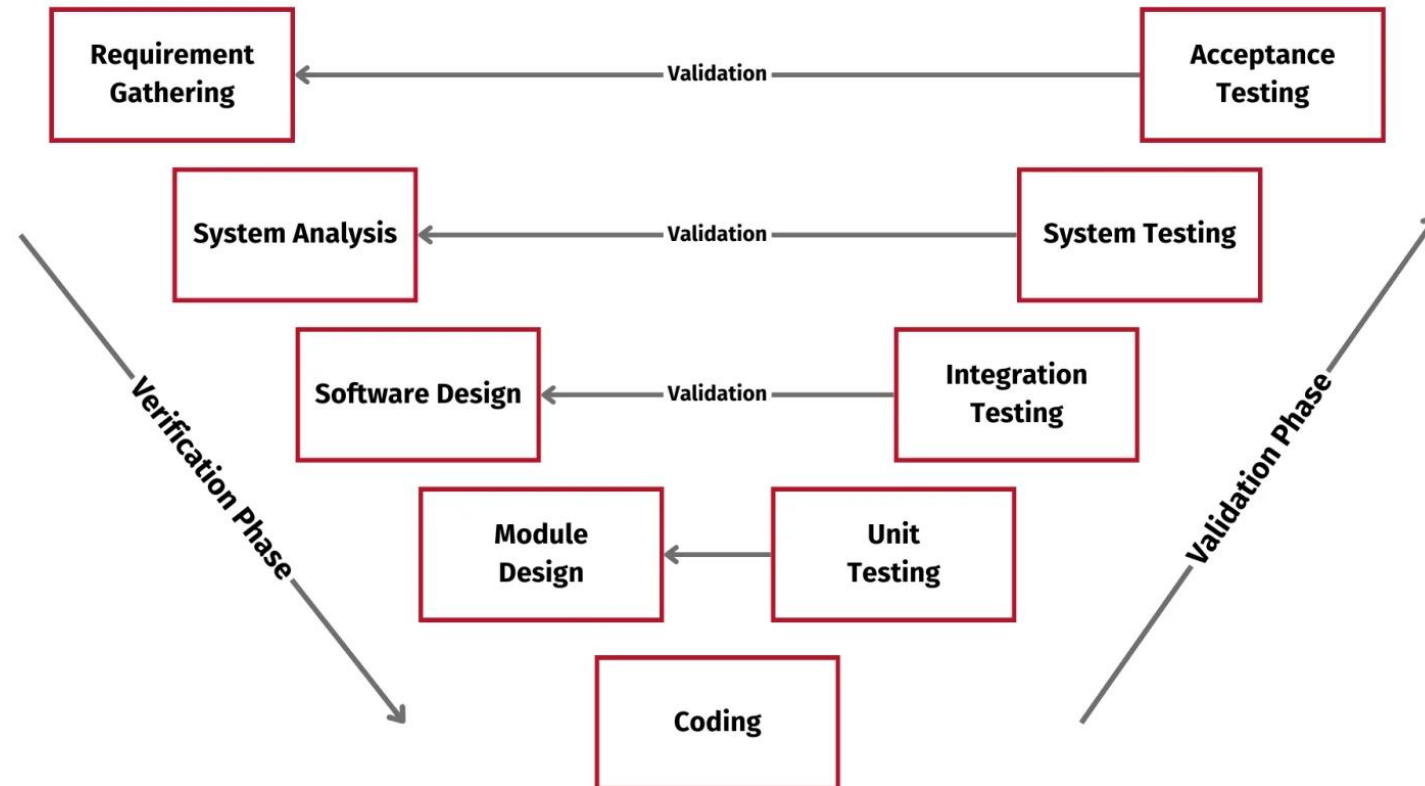
V-Model

Pros:

- Early Tests
- Direct Traceability
- Clear Governance

Cons:

- High Inflexibility
- Late Working Software
- Poor Fit for Ambiguity



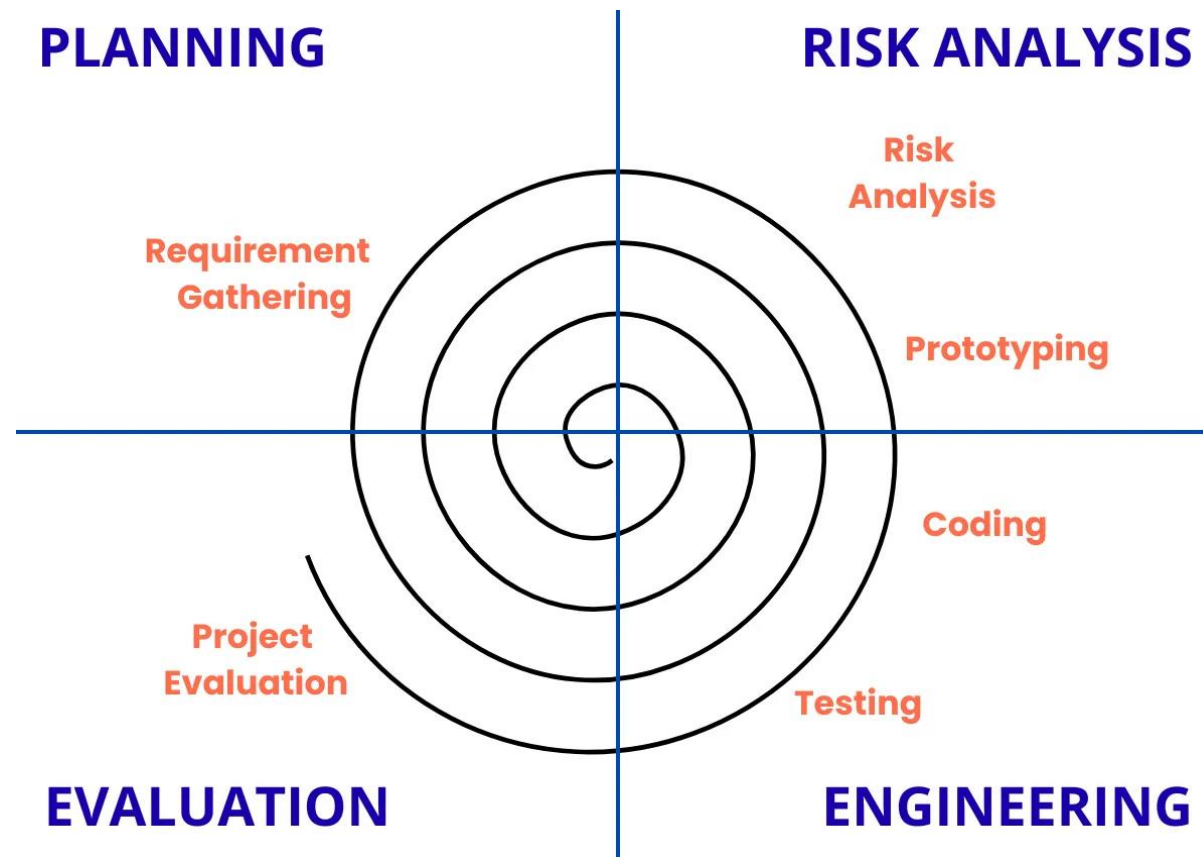
Spiral Model

Pros:

- Proactive Risk Mitigation
- Incremental Feedback
- High Adaptability

Cons:

- High Overhead, Cost
- Heavy Expertise Dependency
- Oversized for Standard Projects



Next Lecture

What's Next:

